

Trabalho 1 — Contagem de Operações e de Tempo de Execução

Estruturas de Dados I — ICMC-USP — 2026

Integrantes:

- João Pedro Oliveira — 17930847
 - Matheus Amaral Alves — 17932255
 - Lorenzo Vittorio Rudnik Spelta — 17906700
 - Gustavo Cestari Cassaro Mendes — 17990719
-

1. Introdução

Este trabalho consiste em implementar e analisar quatro algoritmos que operam sobre um vetor ordenado de inteiros: inversão da ordem do vetor, busca sequencial, busca binária iterativa e busca binária recursiva. Para cada um, o objetivo é medir experimentalmente o tempo de execução e a quantidade de operações (comparações e atribuições) no pior caso, além de determinar analiticamente as funções de custo $T(n)$ e a complexidade assintótica nos casos melhor, médio e pior.

Foi implementado um programa em C, sem bibliotecas externas, com um pequeno menu que executa as quatro operações sobre um vetor de entrada, seguindo o formato de entrada/saída especificado no enunciado (esse é o programa submetido no Run.Codes). As quatro funções ficam em um arquivo separado (`algoritmos.c`), compartilhado entre o programa do menu e o código auxiliar de medição, de modo que o código medido é exatamente o código entregue.

Separadamente, foi desenvolvido um código auxiliar de benchmark (`benchmark.c`) que lê vetores ordenados gerados por `gerador.c` nos tamanhos exigidos (1.000, 50.000, 100.000, 500.000 e 1.000.000), executa cada algoritmo 100 vezes de forma sequencial para obter o tempo médio de execução, e conta as operações realizadas no pior caso (busca por um valor ausente do vetor). Os resultados são gravados em arquivos `.csv` e os gráficos são gerados por scripts em Python (`grafico.py` e `grafico_medicoes.py`).

O relatório segue a estrutura pedida no enunciado: a Seção 2 traz a análise empírica (metodologia, tabelas e gráficos por algoritmo), a Seção 3 a análise assintótica teórica, a Seção 4 a comparação entre os quatro algoritmos e a Seção 5 a contribuição de cada integrante.

2. Análise Empírica

2.1 Metodologia

Ambiente. Todas as medições foram feitas na mesma máquina: processador Intel Core Ultra 7 258V (8 núcleos), 30 GB de RAM, Linux (kernel 7.1), compilador gcc 16.2.1, com o perfil de energia do sistema em `performance` durante o benchmark (o perfil `power-saver` do notebook limita a frequência e deixou os tempos cerca de 2,7× maiores em uma rodada de controle, sem alterar nenhuma contagem). O código foi compilado com `gcc -O2 -std=c99 -Wall -Wextra`. Todos os números de tempo deste relatório vêm de uma única rodada, feita após a integração das funções de todos os integrantes; medições feitas individualmente em outras máquinas foram usadas apenas para validação das contagens.

Dados de entrada. Os vetores foram gerados por `gerador.c` com semente fixa (`srand(42)`), para que qualquer integrante reproduza exatamente os mesmos dados. Cada vetor é estritamente crescente: o primeiro elemento é um valor aleatório em $[0, 50]$ e cada elemento seguinte é o anterior somado a um incremento aleatório em $[1, 50]$. Isso garante que o vetor está ordenado (restrição do enunciado) e que não há elementos repetidos. Os arquivos `dados/dados_<n>.txt` contêm apenas n na primeira linha e o vetor na segunda.

Medição de tempo. Antes de medir cada tamanho, o benchmark mantém o processador ocupado por 0,3 s, para que o governador de frequência leve o núcleo ao clock máximo e todos os tamanhos sejam medidos na mesma frequência (sem isso, os tamanhos pequenos — que são medidos primeiro e terminam em milissegundos — rodavam em clock mais baixo que os grandes, ver Seção 2.2). Em seguida, para cada algoritmo, executa a função 3 vezes sem cronometrar (aquecimento de cache, descartado) e depois 100 vezes cronometrando cada chamada individualmente com `clock_gettime(CLOCK_MONOTONIC)`, que tem resolução de nanossegundos. O tempo médio reportado é a média aritmética dessas 100 medições. Os tamanhos são medidos um após o outro, no mesmo processo, fixado em um único núcleo (`taskset`, pois o processador tem núcleos de desempenho e de eficiência com clocks diferentes), sem outros processos do benchmark concorrendo pela CPU. Para o `inverter`, a mesma cópia do vetor é reinvertida a cada chamada — o custo de inverter um vetor não depende dos valores armazenados, então isso não altera a medição.

Contagem de operações. A contagem é feita em uma execução separada, com os contadores globais `g_comparacoes` e `g_atribuicoes` zerados imediatamente antes da chamada. A convenção do que é contado está detalhada na Seção 3.1. Os incrementos dos contadores fazem parte do código medido. Com `-O2`, o gcc mantém os dois contadores em registradores durante o laço e escreve na memória uma única vez, no retorno da função; o custo adicional é uma soma por operação contada, igual nos quatro algoritmos, e não altera as conclusões comparativas.

Pior caso das buscas. Em todas as buscas o valor procurado é $v[n-1] + 1$, que é maior que todos os elementos do vetor e portanto está ausente. Para a busca sequencial, isso força a varredura de todos os n elementos antes de concluir que o valor não existe. Para as buscas binárias, um valor maior que todos os elementos faz a busca descer sempre pela metade direita até o intervalo ficar vazio, o que produz o número máximo de iterações ($\lfloor \log_2 n \rfloor + 1$, ver Seção 3.4). Para o `inverter` não há distinção de casos: o trabalho depende apenas de n .

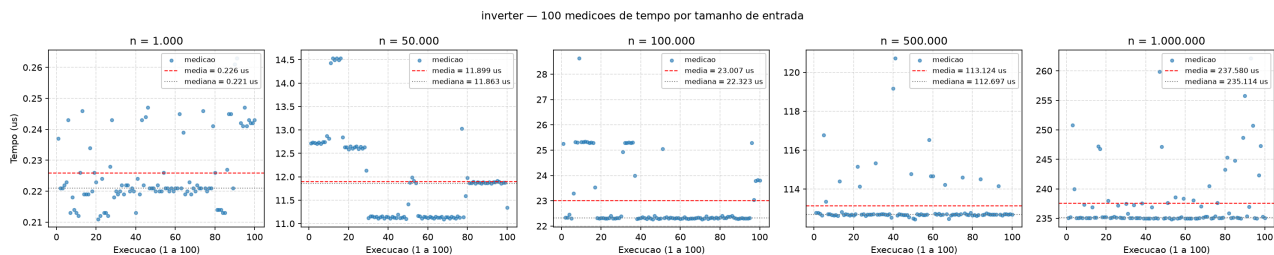
Arquivos gerados.

- `resultados/resultado_completo.csv` — tempo médio, comparações e atribuições no pior caso, para cada algoritmo e tamanho;
- `resultados/tempos_detalhados.csv` — as 100 medições individuais de tempo de cada algoritmo e tamanho;
- `graficos/<algoritmo>_100_medicoes.png` — um gráfico por algoritmo, com um painel por tamanho mostrando as 100 medições e a média;
- `graficos/tempo_medio.png` (escala log-log), `graficos/comparacoes.png` e `graficos/atribuicoes.png` — comparação entre os quatro algoritmos, usados na Seção 4.

2.2 Inversão do Vetor (inverter)

n	tempo médio (s)	comparações (medido)	comparações (teórico)	atribuições (medido)	atribuições (teórico)
1.000	0,000000226	501	501	2.502	2.502
50.000	0,000011899	25.001	25.001	125.002	125.002
100.000	0,000023007	50.001	50.001	250.002	250.002
500.000	0,000113124	250.001	250.001	1.250.002	1.250.002
1.000.000	0,000237580	500.001	500.001	2.500.002	2.500.002

O gráfico abaixo mostra as 100 medições de tempo de cada tamanho separadamente (um painel por tamanho; a linha tracejada é a média):



Discussão. O número de comparações e de atribuições medido coincide exatamente com o previsto pela análise teórica da Seção 3.2 ($\lfloor n/2 \rfloor + 1$ comparações e $5\lfloor n/2 \rfloor + 2$ atribuições): a inversão não possui casos distintos, pois o número de trocas depende apenas de n , nunca dos valores armazenados no vetor. As duas contagens crescem linearmente com n — dobrar o tamanho do vetor dobra o número de operações — e o tempo médio acompanha esse comportamento, como esperado para um algoritmo $O(n)$. O custo por elemento é pequeno e constante: cada iteração realiza uma troca de dois inteiros e avança dois índices, sem chamadas de função nem alocação de memória.

Nos tempos, as razões entre tamanhos consecutivos confirmam a linearidade com precisão: de 50.000 para 100.000 o tempo cresce 1,93×; de 100.000 para 500.000, 4,92×; de 500.000 para 1.000.000, 2,10×; e de 1.000 para 1.000.000 (1000× mais elementos), 1051×. O custo por elemento é praticamente constante em todos os tamanhos, cerca de 0,23–0,24 ns — ou seja, o tempo é bem descrito por $T(n) \approx 0,23 \cdot n$ ns, uma reta de inclinação 1 no gráfico log-log da Seção 4. O fato de o custo por elemento não aumentar em $n = 1.000.000$ (vetor de 4 MB, que já não cabe na cache L2) indica que o acesso sequencial pelos dois extremos do vetor é bem servido pelo *prefetch* do processador, e a inversão não chega a ser limitada pela memória.

Um problema de medição encontrado durante o trabalho: em uma rodada anterior, sem o aquecimento global de 0,3 s descrito na Seção 2.1, o custo por elemento parecia cair pela metade entre 100.000 e 500.000. A causa não era o algoritmo: os tamanhos são medidos em ordem crescente e os pequenos terminam em poucos milissegundos, antes de o governador de frequência elevar o clock do núcleo; os grandes já rodavam com o núcleo na frequência máxima. Manter o processador ocupado por alguns décimos de segundo antes de cada tamanho eliminou o efeito.

No gráfico das 100 medições, os tamanhos 500.000 e 1.000.000 têm um platô estável (medianas de 112,7 μ s e 235,1 μ s; em 1.000.000 a mediana fica a menos de 0,1 % do mínimo, 234,9 μ s), com picos isolados de até 1,1–1,25× o valor típico. Os picos são interferência do sistema operacional (trocas de contexto, interrupções), não do algoritmo, que executa o mesmo número de operações em todas as chamadas. Como os picos são sempre para cima, a média fica um pouco acima da mediana (237,6 μ s contra 235,1 μ s em 1.000.000, 1,0 %).

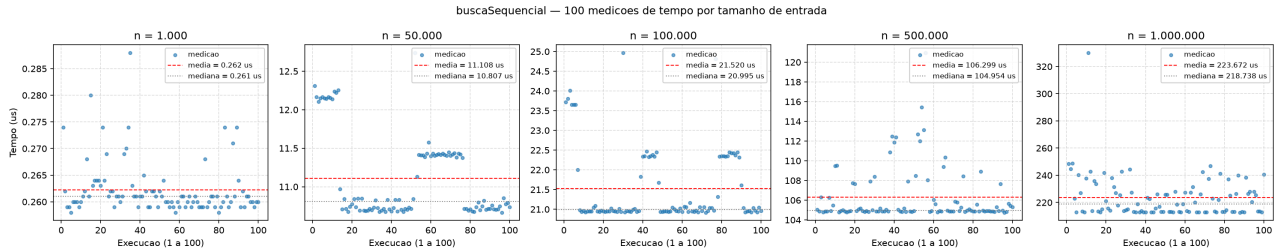
Em 50.000 e 100.000 o comportamento é diferente: as medições formam patamares, isto é, blocos de execuções consecutivas em um nível constante. Em 50.000 há cinco níveis (12,7, 14,5 e 12,6 μ s nas execuções 1–29, 11,1 μ s nas execuções 30–76 e 11,9 μ s nas últimas 22), com amplitude de 30 %; a mediana, 11,86 μ s, cai sobre o patamar

final. Em 100.000 o platô é de 22,3 μ s, com dois blocos em 25,3 μ s (execuções 7–16 e 31–36) que deixam a média 3,1 % acima da mediana. Blocos de execuções consecutivas no mesmo nível deslocado não são picos isolados: indicam mudança de frequência do núcleo ou outro processo disputando o mesmo núcleo naquele trecho. As 100 chamadas desses dois tamanhos duram cerca de 1 e 2 ms no total, uma janela em que uma única mudança de estado do processador afeta dezenas de chamadas seguidas. Nos tamanhos maiores cada chamada dura mais, e a mesma variação aparece diluída como picos individuais. As médias de 50.000 e 100.000 devem ser lidas com essa ressalva; ainda assim, as razões entre tamanhos consecutivos ficaram dentro de 5 % dos valores lineares (2, 5 e 2).

Em $n = 1.000$ a dispersão relativa é a maior das cinco (0,21 a 0,26 μ s): em uma medição de poucas centenas de nanossegundos, o próprio custo da chamada de `clock_gettime` (dezenas de nanossegundos) é uma fração visível do total, e por isso a média desse tamanho é a menos confiável em termos absolutos. Mesmo assim, a razão para $n = 1.000.000$ ficou a cerca de 5 % do esperado.

2.3 Busca Sequencial (buscaSequencial)

n	tempo médio (s)	comparações (medido)	comparações (teórico)	atribuições (medido)	atribuições (teórico)
1.000	0,000000262	2.001	2.001	1.001	1.001
50.000	0,000011108	100.001	100.001	50.001	50.001
100.000	0,000021520	200.001	200.001	100.001	100.001
500.000	0,000106299	1.000.001	1.000.001	500.001	500.001
1.000.000	0,000223672	2.000.001	2.000.001	1.000.001	1.000.001



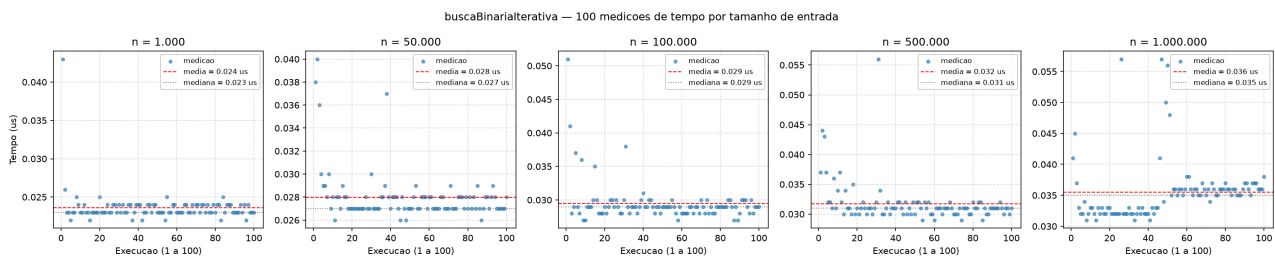
Discussão. As contagens medidas coincidem exatamente com as teóricas da Seção 3.3 ($2n + 1$ comparações e $n + 1$ atribuições): no pior caso — valor ausente — a busca sequencial percorre o vetor inteiro, independentemente dos valores armazenados, e cada elemento custa duas comparações ($i < n$ e $v[i] = x$) e uma atribuição ($i++$).

O tempo médio cresce linearmente com n : de 50.000 para 100.000, 1,94 $\times$; de 100.000 para 500.000, 4,94 $\times$; de 500.000 para 1.000.000, 2,10 $\times$. O custo por elemento fica em torno de 0,21–0,22 ns (em $n = 1.000$ sobe para 0,26 ns por causa do custo fixo da leitura do relógio, que pesa mais em uma chamada de 0,26 μ s), e no gráfico log-log da Seção 4 a curva é uma reta de inclinação 1, praticamente sobreposta à do `inverter`. A busca sequencial faz cerca de 4 $\times$ mais comparações que o `inverter` para o mesmo n ($2n + 1$ contra $\lfloor n/2 \rfloor + 1$), mas em tempo ficou um pouco **mais rápida** (223,7 μ s contra 237,6 μ s em $n = 1.000.000$, razão de 0,94). A contagem de operações e o tempo medem coisas diferentes. A busca só lê o vetor, em uma varredura contígua que o `prefetch` da cache serve com eficiência, e sua comparação $v[i] = x$ é sempre falsa no pior caso, o que o preditor de desvios do processador acerta em todas as iterações. Já o `inverter` lê e **escreve** duas posições por iteração, em dois pontos distantes do vetor, o que gera mais tráfego de memória por elemento visitado. Por isso a contagem de operações é a métrica usada para classificar algoritmos (ambos são $O(n)$, com constantes 2 e 1/2 em comparações); o tempo absoluto depende de detalhes da arquitetura que a análise assintótica ignora de propósito.

Nas 100 medições individuais, 500.000 e 1.000.000 apresentam um platô estável (medianas de 104,95 μ s e 218,7 μ s, esta com mínimo de 212,7 μ s) e picos isolados causados por interferência do sistema operacional — o maior deles, em $n = 1.000.000$, de 330 μ s ($1,5\times$ o valor típico), suficiente para puxar a média 2,3 % acima da mediana. Em 50.000 e 100.000 as medianas são 10,81 μ s e 21,00 μ s. Em 50.000 e 100.000 aparecem os mesmos patamares vistos no `inverter`: em 50.000, as execuções 1–13 ficam em 12,15 μ s e as execuções 54–76 em 11,4 μ s, contra um platô de 10,7 μ s; em 100.000, as execuções 1–7 ficam em 23,7 μ s e dois blocos (39–47 e 79–89) em 22,4 μ s, contra 21,0 μ s. Como os blocos aparecem também no meio da rodada, não é efeito de aquecimento; é a mesma variação de estado do processador discutida na Seção 2.2, que afeta os tamanhos medidos em poucos milissegundos.

2.4 Busca Binária Iterativa (`buscaBinariaIterativa`)

n	tempo médio (s)	comparações (medido)	comparações (teórico)	atribuições (medido)	atribuições (teórico)
1.000	0,000000024	31	31	22	22
50.000	0,000000028	49	49	34	34
100.000	0,000000029	52	52	36	36
500.000	0,000000032	58	58	40	40
1.000.000	0,000000036	61	61	42	42



Discussão. As contagens medidas coincidiram com as teóricas em todos os tamanhos: sendo $k = \lfloor \log_2 n \rfloor + 1$, a função realiza $3k + 1$ comparações e $2k + 2$ atribuições no pior caso utilizado. O alvo foi $v[n-1] + 1$, ausente e maior que todos os elementos, de modo que a busca sempre segue pela metade direita até o intervalo ficar vazio. Ao passar de 1.000 para 1.000.000 de elementos, o número de iterações aumenta de 10 para 20; as comparações passam de 31 para 61 e as atribuições, de 22 para 42. Esse crescimento confirma a função de custo logarítmica.

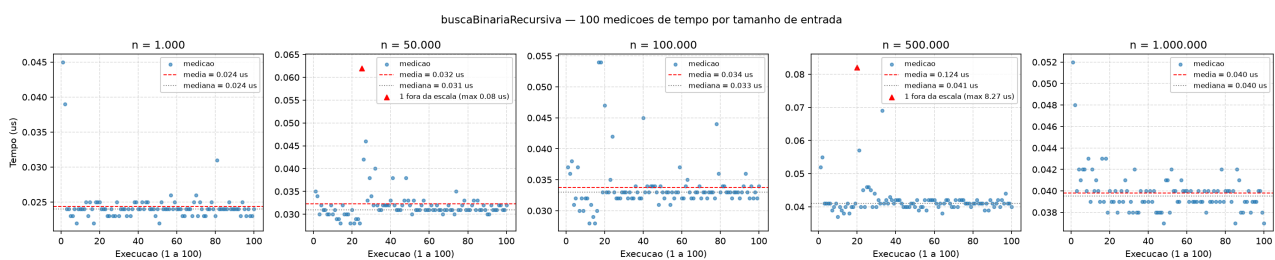
Os tempos médios foram 24, 28, 30, 32 e 36 ns para os cinco tamanhos (medianas de 23, 27, 29, 31 e 35 ns) — cada chamada completa em algumas dezenas de nanossegundos mesmo para um milhão de elementos, contra 223,7 μ s da busca sequencial no mesmo vetor (uma diferença de cerca de $6.200\times$). O crescimento do tempo com n é pequeno, mas coerente com o número de iterações: 10 iterações em cerca de 24 ns e 20 em cerca de 36 ns. O salto entre 500.000 e 1.000.000 (32 $\rightarrow$ 36 ns) é maior do que uma iteração a mais justificaria, e o gráfico das 100 medições mostra a causa: em 1.000.000, as execuções 1–45 têm mediana de 32 ns, igual à de 500.000, e as execuções 46–100 sobem para 36 ns. É um degrau no meio da rodada, do mesmo tipo dos patamares da Seção 2.2, e não uma propriedade do algoritmo. O tamanho do vetor não entra nessa conta: a busca acessa as mesmas ~ 20 posições em todas as chamadas, e depois das chamadas de aquecimento essas linhas já estão na cache.

Nessa escala, o custo da leitura do relógio (`clock_gettime`, da ordem de 20 ns) e a instrumentação dos contadores são uma fração relevante do tempo medido, o que se reflete no gráfico das 100 medições: os pontos se concentram em dois ou três patamares separados por 1 ns (granularidade do relógio), com alguns picos isolados de até 43–57 ns (cerca de $1,6$ – $1,9\times$ a mediana), concentrados nas primeiras execuções de cada tamanho. Por isso a diferença entre as médias dos tamanhos deve ser lida qualitativamente. A evidência sólida do comportamento

$O(\log n)$ é a contagem determinística de operações, que reproduz exatamente $3k + 1$ e $2k + 2$ em todos os tamanhos.

2.5 Busca Binária Recursiva (buscaBinariaRecursiva)

n	tempo médio (s)	comparações (medido)	comparações (teórico)	atribuições (medido)	atribuições (teórico)
1.000	0,000000024	31	31	10	10
50.000	0,000000032	49	49	16	16
100.000	0,000000034	52	52	17	17
500.000	0,000000124	58	58	19	19
1.000.000	0,000000040	61	61	20	20



Discussão. As contagens medidas coincidem exatamente com as teóricas da Seção 3.5: sendo $k = \lfloor \log_2 n \rfloor + 1$ o número de chamadas "cheias" (que calculam `meio`), a função realiza $3k + 1$ comparações e k atribuições no pior caso. As comparações são idênticas às da versão iterativa (31, 49, 52, 58 e 61), pois as duas implementações percorrem exatamente a mesma sequência de intervalos: a recursão apenas substitui o laço `while` por uma chamada de função e a atualização de `ini/fim` por passagem de parâmetro. É essa substituição que explica as atribuições menores (10 contra 22 em $n = 1.000$; 20 contra 42 em $n = 1.000.000$): pela convenção da Seção 3.1, passagem de parâmetro não conta como atribuição, e a única variável atribuída em cada chamada é `meio`. É uma diferença de convenção de contagem, não de trabalho realizado — o processador ainda precisa colocar os novos limites em registradores a cada passo.

Em tempo, a versão recursiva ficou um pouco mais lenta que a iterativa: médias de 24, 32, 34 e 40 ns em $n = 1.000, 50.000, 100.000$ e $1.000.000$ (medianas de 24, 31, 33 e 39,5 ns), contra 24, 28, 30 e 36 ns da iterativa. A diferença é de cerca de 4 ns em 50.000, 100.000 e 1.000.000 (11 % pelas médias em 1.000.000) e de menos de 1 ns em 1.000.

A explicação esperada seria o custo de cada chamada de função (empilhar endereço de retorno e parâmetros a cada nível). Não é o que acontece no binário medido. Inspecionando o assembly gerado por `gcc -O2 -S, bbRec` foi inlinada em `buscaBinariaRecursiva` e as duas chamadas recursivas, que estão em posição de cauda (`return bbRec(...)`), foram convertidas em um salto para o início da função: não há nenhuma instrução `call` no código gerado, e a recursão vira um laço equivalente ao da versão iterativa. Os 4 ns a mais vêm de diferenças no código gerado para esse laço (um registrador a mais salvo na pilha na entrada e algumas movimentações extras entre registradores), não do número de níveis — o que bate com a diferença ser praticamente constante entre 50.000 e 1.000.000 em vez de crescer com $\log_2 n$. A sobrecarga de chamada e a pilha $O(\log n)$ previstas na Seção 3.5 só apareceriam compilando sem otimização (`-O0`). Nos dois casos a diferença é de constante e não altera a classe $O(\log n)$.

O caso de $n = 500.000$ exige um comentário à parte. A média registrada, 124 ns, é três vezes a mediana (41 ns) e destoia da tendência dos demais tamanhos. A causa é uma única medição, de 8,27 μs — cerca de 200 vezes o valor típico —, correspondente a uma interrupção do sistema operacional que caiu dentro de uma chamada de 40 ns; as

outras 99 medições ficaram entre 37 e 69 ns. No gráfico das 100 medições esse ponto aparece como um marcador vermelho "fora da escala", com o valor máximo anotado, e a linha pontilhada da mediana mostra onde está o custo típico da chamada. Para chamadas de dezenas de nanossegundos, a mediana é um estimador mais robusto que a média: um único evento externo, irrelevante para o algoritmo, basta para triplicar a média de 100 execuções. É esse ponto que produz o salto da curva da recursiva em $n = 500.000$ no gráfico de tempo médio da Seção 4. Mesmo pela mediana, porém, o tamanho 500.000 fica fora da tendência: 41 ns, acima dos 39,5 ns de 1.000.000. As 99 medições restantes estão em um nível um pouco deslocado para cima durante toda a rodada, do mesmo tipo dos patamares descritos na Seção 2.2. Em 50.000 há um segundo ponto marcado fora da escala no gráfico (77 ns, $2,5\times$ a mediana), sem efeito relevante na média.

Fora isso, a leitura dos tempos é a mesma da versão iterativa: chamadas de dezenas de nanossegundos, com o custo da leitura do relógio pesando de forma relevante, e crescimento suave com o número de níveis. A evidência sólida do comportamento $O(\log n)$ continua sendo a contagem determinística de operações, que reproduz $3k + 1$ e k em todos os tamanhos.

3. Análise Assintótica Teórica

3.1 Convenção de contagem

Para que as contagens dos quatro algoritmos sejam comparáveis entre si, adotou-se uma única convenção, aplicada tanto na análise teórica quanto na instrumentação do código:

- **Comparação:** toda avaliação de uma condição de laço ou de `if` que envolva índices ou elementos do vetor (por exemplo `i < j`, `ini ≤ fim`, `v[meio] = x`, `v[meio] < x`). A comparação que avalia falso e encerra um laço também é contada.
- **Atribuição:** toda escrita em uma posição do vetor ou em uma variável de controle do algoritmo, incluindo inicializações e atualizações de índices (`i = 0`, `j = n - 1`, `i++`, `j--`, `meio = (ini + fim) / 2`, `ini = meio + 1`, `fim = meio - 1`) e a variável temporária da troca (`temp = v[i]`).
- **Não contados:** a passagem de parâmetros em chamadas de função, o `return` e as operações aritméticas em si (`ini + fim`, `/ 2`).

A escolha de contar também as variáveis de controle é deliberada: se apenas as escritas no vetor fossem contadas, as buscas teriam zero atribuições (elas nunca escrevem em `v`) e a coluna de atribuições deixaria de trazer qualquer informação sobre elas. Com a convenção adotada, a contagem de atribuições reflete o trabalho de "caminhar" pelo vetor, que é justamente o que diferencia as buscas.

Em todas as análises, n é o número de elementos do vetor e x o valor buscado. $\log_2$ denota o logaritmo na base 2 e $\lfloor \cdot \rfloor$ a função piso.

3.2 Inversão do Vetor (inverter)

```
void inverter(int v[], int n) {
    int i = 0;           // atrib
    int j = n - 1;       // atrib
    while (i < j) {      // comp (uma por iteração + a final que falha)
        int temp = v[i]; // atrib
        v[i] = v[j];     // atrib
        v[j] = temp;     // atrib
        i++;             // atrib
        j--;             // atrib
    }
}
```

O laço executa enquanto $i < j$. Como i começa em 0 e j em $n - 1$, e a cada iteração os dois se aproximam em 1, o laço faz exatamente $\lfloor n/2 \rfloor$ iterações (para n ímpar o elemento central não é trocado). Cada iteração custa 1 comparação e 5 atribuições; antes do laço há 2 atribuições; e ao final há mais 1 comparação, que avalia falso e encerra o laço.

O número de iterações depende **apenas de n** , nunca dos valores armazenados no vetor: não existe entrada de tamanho n que faça o algoritmo executar mais ou menos trocas. Por isso, melhor caso, caso médio e pior caso são idênticos.

Caso	Comparações T(n)	Atribuições T(n)	Termo dominante	Complexidade
Melhor caso	$\lfloor n/2 \rfloor + 1$	$5\lfloor n/2 \rfloor + 2$	n	$O(n)$
Caso médio	$\lfloor n/2 \rfloor + 1$	$5\lfloor n/2 \rfloor + 2$	n	$O(n)$
Pior caso	$\lfloor n/2 \rfloor + 1$	$5\lfloor n/2 \rfloor + 2$	n	$O(n)$

Conclusão: inverter é $O(n)$ em qualquer cenário (e também $\Omega(n)$, portanto $\Theta(n)$).

3.3 Busca Sequencial (buscaSequencial)

```
int buscaSequencial(int v[], int n, int x) {
    int i = 0;           // atrib
    while (i < n) {      // comp
        if (v[i] == x) { // comp
            return i;
        }
        i++;             // atrib
    }
    return -1;
}
```

Cada iteração completa custa 2 comparações ($i < n$ e $v[i] == x$) e 1 atribuição ($i++$); há 1 atribuição inicial ($i = 0$).

- **Melhor caso:** x está na primeira posição ($x = v[0]$). O laço entra uma vez, faz as 2 comparações e retorna: 2 comparações e 1 atribuição, independentemente de n .
- **Pior caso:** x não está no vetor (ou está na última posição). O laço executa as n iterações completas, cada uma com 2 comparações e 1 atribuição, e então a comparação $i < n$ falha mais uma vez: $2n + 1$ comparações e $n + 1$ atribuições.
- **Caso médio:** supondo que x está no vetor com igual probabilidade em qualquer uma das n posições, se $x = v[k]$ o laço faz $k + 1$ iterações. O número médio de iterações é $(1 + 2 + \dots + n)/n = (n + 1)/2$, o que dá

aproximadamente $n + 1$ comparações e $(n + 1)/2$ atribuições (contando $i = 0$ e as atualizações $i++$ das iterações que não retornaram).

Caso	Comparações T(n)	Atribuições T(n)	Termo dominante	Complexidade
Melhor caso	2	1	1	$O(1)$
Caso médio	$\approx n + 1$	$\approx (n + 1)/2$	n	$O(n)$
Pior caso	$2n + 1$	$n + 1$	n	$O(n)$

Conclusão: `buscaSequencial` é $O(n)$ no pior caso e no caso médio, e $O(1)$ no melhor caso. Observação: como o vetor é ordenado, a busca poderia parar assim que encontrasse $v[i] > x$; essa otimização não altera o pior caso analisado aqui (valor maior que todos os elementos), que continua a exigir a varredura completa.

3.4 Busca Binária Iterativa (`buscaBinariaIterativa`)

```
int buscaBinariaIterativa(int v[], int n, int x) {
    int ini = 0;           // atrib
    int fim = n - 1;       // atrib
    while (ini ≤ fim) {    // comp
        int meio = (ini + fim) / 2; // atrib
        if (v[meio] == x) { // comp
            return meio;
        }
        if (v[meio] < x) { // comp
            ini = meio + 1; // atrib
        } else {
            fim = meio - 1; // atrib
        }
    }
    return -1;
}
```

Cada iteração completa (que não encontra x) custa 3 comparações ($ini \leq fim$, $v[meio] == x$, $v[meio] < x$) e 2 atribuições ($meio$ e um dos limites). Antes do laço há 2 atribuições.

De onde vem $\lfloor \log_2 n \rfloor + 1$. Seja t o tamanho do intervalo $[ini, fim]$ em uma iteração ($t = fim - ini + 1$). Depois de comparar com o elemento central, o intervalo passa a ter $\lfloor (t - 1)/2 \rfloor$ elementos (metade esquerda) ou $\lceil (t - 1)/2 \rceil = \lfloor t/2 \rfloor$ elementos (metade direita). No pior caso a busca segue sempre a metade maior, $\lfloor t/2 \rfloor$. Começando com $t = n$, a sequência de tamanhos é n , $\lfloor n/2 \rfloor$, $\lfloor n/4 \rfloor$, ..., 1, 0; o número de termos não nulos — isto é, o número de iterações executadas — é $\lfloor \log_2 n \rfloor + 1$. É exatamente o que acontece com o alvo $v[n-1] + 1$ usado no benchmark: como ele é maior que todos os elementos, a comparação $v[meio] < x$ é sempre verdadeira e a busca segue sempre pela direita. Para os cinco tamanhos do trabalho, $\lfloor \log_2 n \rfloor + 1$ vale 10, 16, 17, 19 e 20.

- **Melhor caso:** x está exatamente na posição central da primeira iteração. Custa 2 comparações ($ini \leq fim$ e $v[meio] == x$) e 3 atribuições (ini , fim , $meio$).
- **Pior caso:** x ausente. São $\lfloor \log_2 n \rfloor + 1$ iterações completas (3 comparações e 2 atribuições cada) mais a comparação final $ini \leq fim$ que falha, e as 2 atribuições iniciais: $3(\lfloor \log_2 n \rfloor + 1) + 1$ comparações e $2(\lfloor \log_2 n \rfloor + 1) + 2$ atribuições.
- **Caso médio:** para x presente em posição uniformemente aleatória, o número médio de iterações é aproximadamente $\log_2 n - 1$ (a maior parte dos elementos está nos níveis mais profundos da "árvore" de divisões), o que dá aproximadamente $3 \log_2 n$ comparações e $2 \log_2 n$ atribuições.

Caso	Comparações T(n)	Atribuições T(n)	Termo dominante	Complexidade
Melhor caso	2	3	1	O(1)
Caso médio	$\approx 3 \log_2 n$	$\approx 2 \log_2 n$	$\log n$	O(log n)
Pior caso	$3(\lfloor \log_2 n \rfloor + 1) + 1$	$2(\lfloor \log_2 n \rfloor + 1) + 2$	$\log n$	O(log n)

Conclusão: `buscaBinariaIterativa` é **O(log n)** no pior caso e no caso médio, e O(1) no melhor caso.

3.5 Busca Binária Recursiva (`buscaBinariaRecursiva`)

```
static int bbRec(int v[], int ini, int fim, int x) {
    if (ini > fim) {                // comp
        return -1;
    }
    int meio = (ini + fim) / 2;      // atrib
    if (v[meio] == x) {             // comp
        return meio;
    }
    if (v[meio] < x) {              // comp
        return bbRec(v, meio + 1, fim, x);
    }
    return bbRec(v, ini, meio - 1, x);
}

int buscaBinariaRecursiva(int v[], int n, int x) {
    return bbRec(v, 0, n - 1, x);
}
```

A versão recursiva percorre exatamente a mesma sequência de intervalos da versão iterativa: cada chamada de `bbRec` corresponde a uma iteração do laço, e a chamada com `ini > fim` corresponde à comparação final que encerra o laço. Portanto o número de comparações é idêntico ao da Seção 3.4. A diferença está nas atribuições: os novos limites são passados como parâmetros, o que não é contado pela convenção adotada, e a única atribuição por chamada é `meio`. A função de custo $T(n)$ pode ser escrita como a recorrência $T(n) = T(\lfloor n/2 \rfloor) + c$, com $T(0) = c$, cuja solução é $\Theta(\log n)$.

- **Melhor caso:** `x` na posição central da primeira chamada: 2 comparações e 1 atribuição.
- **Pior caso:** `x` ausente. São $\lfloor \log_2 n \rfloor + 1$ chamadas completas (3 comparações e 1 atribuição cada) mais a chamada final com `ini > fim` (1 comparação): $3(\lfloor \log_2 n \rfloor + 1) + 1$ comparações e $\lfloor \log_2 n \rfloor + 1$ atribuições.
- **Caso médio:** como na versão iterativa, aproximadamente $\log_2 n$ chamadas, portanto $\approx 3 \log_2 n$ comparações e $\approx \log_2 n$ atribuições.

Caso	Comparações T(n)	Atribuições T(n)	Termo dominante	Complexidade
Melhor caso	2	1	1	O(1)
Caso médio	$\approx 3 \log_2 n$	$\approx \log_2 n$	$\log n$	O(log n)
Pior caso	$3(\lfloor \log_2 n \rfloor + 1) + 1$	$\lfloor \log_2 n \rfloor + 1$	$\log n$	O(log n)

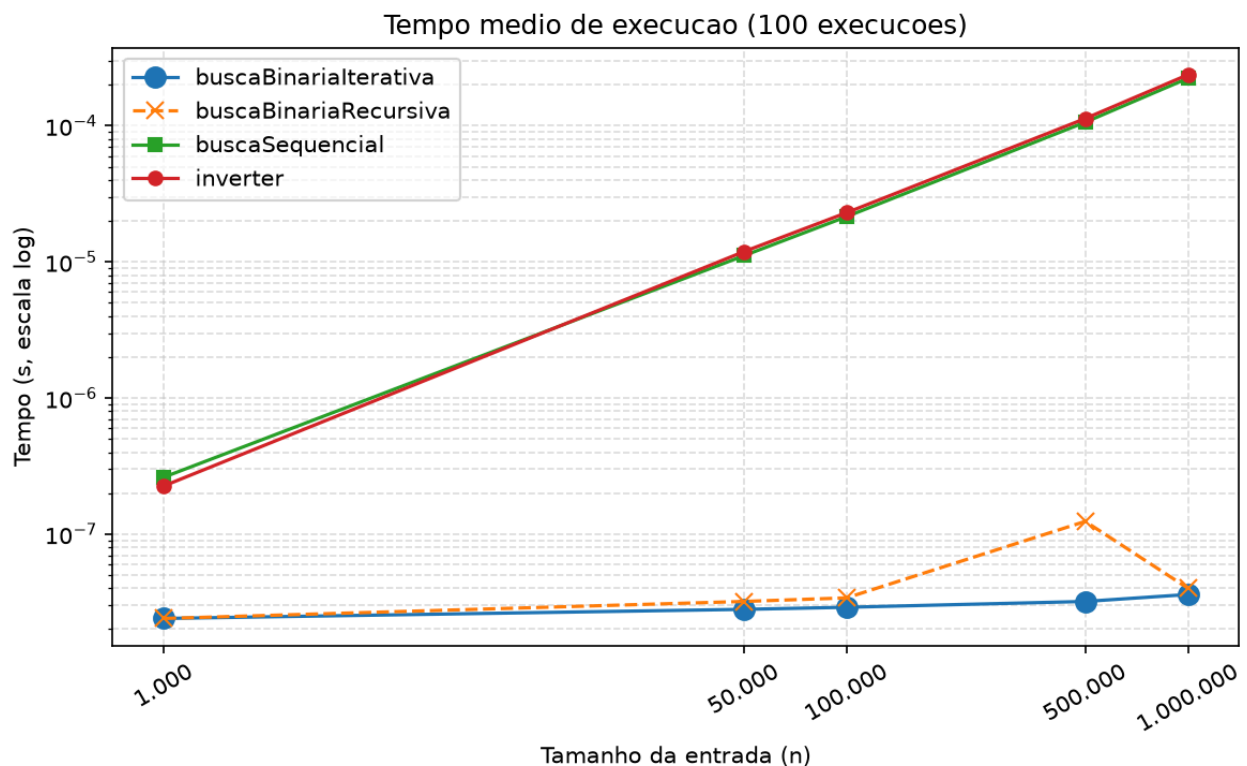
Conclusão: `buscaBinariaRecursiva` é **O(log n)** no pior caso e no caso médio, e O(1) no melhor caso — a mesma classe da versão iterativa. A diferença entre as duas não aparece em $T(n)$: está no custo constante de cada chamada de função e no uso de memória, pois a recursão mantém $\lfloor \log_2 n \rfloor + 2$ quadros na pilha no pior caso (espaço O(log n)), enquanto a versão iterativa usa espaço O(1). Isso vale para o algoritmo como escrito; como as

chamadas recursivas estão em posição de cauda, um compilador com otimização pode convertê-las em salto e eliminar os quadros, que é o que o gcc fez com -O2 (Seção 2.5).

4. Conclusão

4.1 Tempo médio de execução (s)

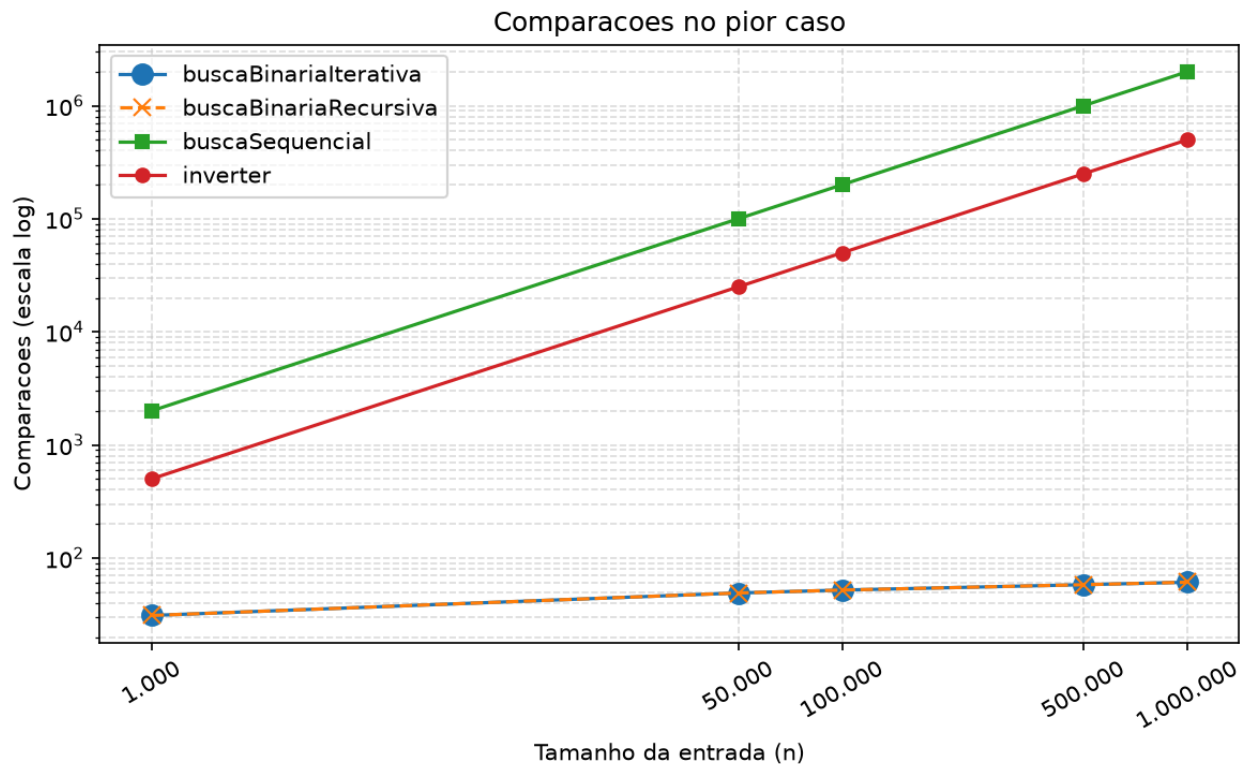
n	inverter	buscaSequencial	buscaBinariaIterativa	buscaBinariaRecursiva
1.000	0,000000226	0,000000262	0,000000024	0,000000024
50.000	0,00011899	0,000011108	0,000000028	0,000000032
100.000	0,00023007	0,000021520	0,000000029	0,000000034
500.000	0,00113124	0,000106299	0,000000032	0,000000124
1.000.000	0,00237580	0,000223672	0,000000036	0,000000040



4.2 Comparações no pior caso

Valores medidos pelo benchmark, que coincidiram com os teóricos da Seção 3 em todos os tamanhos e para os quatro algoritmos.

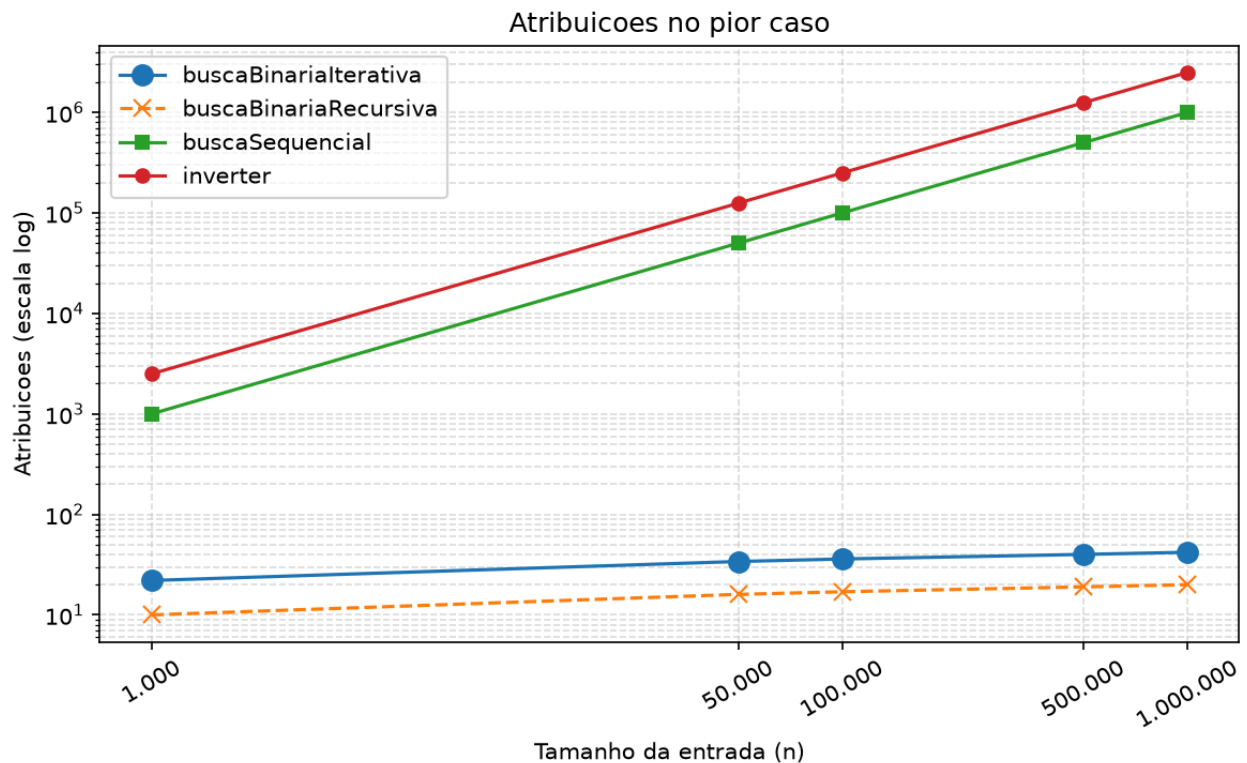
n	inverter	buscaSequencial	buscaBinariaIterativa	buscaBinariaRecursiva
1.000	501	2.001	31	31
50.000	25.001	100.001	49	49
100.000	50.001	200.001	52	52
500.000	250.001	1.000.001	58	58
1.000.000	500.001	2.000.001	61	61



4.3 Atribuições no pior caso

Valores medidos pelo benchmark, que coincidiram com os teóricos da Seção 3 em todos os tamanhos e para os quatro algoritmos.

n	inverter	buscaSequencial	buscaBinariaIterativa	buscaBinariaRecursiva
1.000	2.502	1.001	22	10
50.000	125.002	50.001	34	16
100.000	250.002	100.001	36	17
500.000	1.250.002	500.001	40	19
1.000.000	2.500.002	1.000.001	42	20



4.4 Complexidade assintótica (Big-O)

Algoritmo	Melhor caso	Caso médio	Pior caso
inverter	$O(n)$	$O(n)$	$O(n)$
buscaSequencial	$O(1)$	$O(n)$	$O(n)$
buscaBinariaIterativa	$O(1)$	$O(\log n)$	$O(\log n)$
buscaBinariaRecursiva	$O(1)$	$O(\log n)$	$O(\log n)$

4.5 Discussão

Os gráficos e as tabelas acima separam os quatro algoritmos em dois grupos bem distintos.

Algoritmos lineares. `inverter` e `buscaSequencial` são ambos $O(n)$ no pior caso, e no gráfico log-log de tempo isso aparece como duas retas de inclinação 1, aproximadamente paralelas: multiplicar n por 10 multiplica o tempo por cerca de 10 (para o `inverter`, de 23,0 μs em $n = 100.000$ para 237,6 μs em $n = 1.000.000$, razão de 10,3; para a busca sequencial, de 21,5 μs para 223,7 μs , razão de 10,4). A diferença entre os dois está na constante. A busca sequencial faz cerca de 4 vezes mais comparações que o `inverter` para o mesmo n ($2n + 1$ contra $\lfloor n/2 \rfloor + 1$), porque visita todos os n elementos e realiza duas comparações em cada um, enquanto o `inverter` percorre apenas $n/2$ pares com uma comparação por par. Em atribuições a relação se inverte ($n + 1$ contra $5\lfloor n/2 \rfloor + 2$), pois a inversão escreve no vetor e a busca só avança um índice. Em tempo, porém, a busca sequencial ficou ligeiramente **mais rápida** que a inversão (0,94 $\times$ em $n = 1.000.000$), apesar de fazer 4 $\times$ mais comparações: ela apenas lê o vetor, em varredura contígua e com um desvio perfeitamente previsível, enquanto a inversão lê e escreve em dois pontos distantes a cada iteração. A contagem de operações classifica os algoritmos (ambos $O(n)$); a constante de tempo real depende da arquitetura.

Algoritmos logarítmicos. As duas buscas binárias são indistinguíveis em número de comparações — a versão recursiva percorre exatamente os mesmos intervalos da iterativa — e ambas ficam mais de quatro ordens de

grandeza abaixo da busca sequencial: em $n = 1.000.000$ são 61 comparações contra 2.000.001. O crescimento com n é tão lento que quase não aparece nos gráficos: multiplicar n por 1.000 (de 1.000 para 1.000.000) apenas dobra o número de iterações, de 10 para 20. A diferença entre as duas versões está no tempo, não nas operações: a recursiva ficou cerca de 4 ns (11 %) mais lenta que a iterativa em $n = 1.000.000$ (40 contra 36 ns pelas médias). Como mostrado na Seção 2.5, com -O2 o compilador eliminou a recursão (as chamadas são de cauda e viraram um salto), então a diferença vem do código gerado para o laço, não do custo de empilhar chamadas. É uma diferença de constante, que não muda a classe $O(\log n)$. O ponto da recursiva em $n = 500.000$ no gráfico de tempo médio, visivelmente fora da reta, não contradiz isso: como discutido na Seção 2.5, ele resulta de uma única medição de 8,27 μ s (uma interrupção do sistema) que triplicou a média de 100 execuções cuja mediana é de 41 ns.

Tempo versus contagem de operações. Uma chamada de busca binária em um vetor de um milhão de elementos leva de 36 a 40 ns, valor próximo da resolução prática do relógio e do custo da própria chamada a `clock_gettime`. Nessa escala, o tempo medido reflete tanto o algoritmo quanto o ruído do sistema (escalonamento, cache, variação de frequência), o que explica a maior dispersão relativa nas 100 medições das buscas binárias e a sensibilidade da média a um único evento externo. Nos algoritmos lineares o mesmo ruído aparece como patamares em 50.000 e 100.000 e como picos isolados nos tamanhos maiores. A contagem de operações, por outro lado, é determinística e reproduz exatamente as funções $T(n)$ da Seção 3 em todos os tamanhos. Isso justifica o uso da contagem de operações como métrica principal de comparação, com o tempo servindo para confirmar a tendência e para expor os custos constantes (como o overhead de chamada da recursão) que a análise assintótica deliberadamente ignora.

Síntese. Para um vetor ordenado, a busca binária é muito mais eficiente que a sequencial, e a escolha entre iterativa e recursiva é uma questão de constante e de uso de memória (pilha $O(\log n)$ contra $O(1)$ no código sem otimização), não de complexidade. A inversão, por sua vez, é um caso em que a análise assintótica é exata: sem distinção entre melhor, médio e pior caso, o custo medido é literalmente a função $T(n)$ calculada.

5. Contribuição de cada membro

Integrante	Contribuição
João Pedro Oliveira	Implementação da função <code>inverter</code> ; estruturação do projeto (gerador de dados de teste, benchmark de tempo e de operações, scripts de gráficos); estrutura do relatório, metodologia e análise teórica de referência dos quatro algoritmos; integração das quatro funções e rodada única de medições
Matheus Amaral Alves	Implementação da função <code>buscaSequencial</code> e análise empírica correspondente
Lorenzo Vittorio Rudnik Spelta	Implementação da função <code>buscaBinariaIterativa</code> , testes de corretude e análise empírica correspondente
Gustavo Cestari Cassaro Mendes	Implementação da função <code>buscaBinariaRecursiva</code> e análise empírica correspondente